In [8]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix,accuracy_score,precision_score,recall_
from sklearn.preprocessing import StandardScaler
```

In [9]:
```
df = pd.read_csv('/home/backup/ml_datasets/breast_cancer.csv');
print(df.head(5))
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
0                 0.07871  ...          17.33           184.60      2019.0
1                 0.05667  ...          23.41           158.80      1956.0
2                 0.05999  ...          25.53           152.50      1709.0
3                 0.09744  ...          26.50            98.87       567.7
4                 0.05883  ...          16.67           152.20      1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   worst symmetry  worst fractal dimension  target
0          0.4601                  0.11890       0
1          0.2750                  0.08902       0
2          0.3613                  0.08758       0
3          0.6638                  0.17300       0
4          0.2364                  0.07678       0

[5 rows x 31 columns]
```

In [10]:
```
print(df.head(5))
print(df.isnull().sum())
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
```

```
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
0                 0.07871  ...          17.33           184.60      2019.0
1                 0.05667  ...          23.41           158.80      1956.0
2                 0.05999  ...          25.53           152.50      1709.0
3                 0.09744  ...          26.50            98.87       567.7
4                 0.05883  ...          16.67           152.20      1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   worst symmetry  worst fractal dimension  target
0          0.4601                  0.11890       0
1          0.2750                  0.08902       0
2          0.3613                  0.08758       0
3          0.6638                  0.17300       0
4          0.2364                  0.07678       0

[5 rows x 31 columns]
mean radius                0
mean texture               0
mean perimeter             0
mean area                  0
mean smoothness            0
mean compactness           0
mean concavity             0
mean concave points        0
mean symmetry              0
mean fractal dimension     0
radius error               0
texture error              0
perimeter error            0
area error                 0
smoothness error           0
compactness error          0
concavity error            0
concave points error       0
symmetry error             0
fractal dimension error    0
worst radius               0
worst texture              0
worst perimeter            0
worst area                 0
worst smoothness           0
worst compactness          0
worst concavity            0
worst concave points       0
worst symmetry             0
worst fractal dimension    0
target                     0
dtype: int64
```

In [11]:

```
X = df.drop('target',axis = 1)
y = df[['target']]
```

In [12]:

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

print(f"X_train shape: {X_train.shape[0]}")
print(f"X_test shape: {X_test.shape[0]}")
print(f"y_train shape: {y_train.shape[0]}")
print(f"y_test shape: {y_test.shape[0]}")
```

```
X_train shape: 455
X_test shape: 114
y_train shape: 455
y_test shape: 114
```

In [23]:

```
log_reg = LogisticRegression(max_iter=1000,random_state=42)
log_reg.fit(X_train,y_train)
print(f"Intercept: {log_reg.intercept_}")
print(f"Coefficient: {log_reg.coef_}")
y_pred = log_reg.predict(X_test)
```

```
/usr/local/lib64/python3.6/site-packages/sklearn/utils/validation.py:63: DataConversi
onWarning: A column-vector y was passed when a 1d array was expected. Please change t
he shape of y to (n_samples, ), for example using ravel().
  return f(*args, **kwargs)
Intercept: [6.53774977]
Coefficient: [[ 2.06747835e+00  9.84518072e-02 -1.44640319e-01  1.52127098e-03
  -2.31523633e-01 -3.07308383e-01 -7.84695065e-01 -5.51434484e-01
  -2.91631060e-01 -7.79523203e-03 -1.22845403e-01  7.36725542e-01
  -7.33865099e-02 -7.23849801e-02 -3.99058161e-02  1.90377232e-01
   1.12154970e-01 -5.38473419e-02 -3.54262670e-02  3.34436267e-02
   6.75558284e-01 -3.36907598e-01 -6.42511239e-02 -2.12221115e-02
  -4.43175578e-01 -6.18064256e-01 -1.54507525e+00 -8.76361909e-01
  -1.03785669e+00 -4.19804479e-02]]
/usr/local/lib64/python3.6/site-packages/sklearn/linear_model/_logistic.py:765: Conve
rgenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
```

In [24]:

```
print("\n--DETAILS--")
print(f"Accuracy:{accuracy_score(y_test, y_pred ):.4f}")
print(f"Precision:{precision_score(y_test, y_pred ):.4f}")
print(f"Recall:{recall_score(y_test, y_pred ):.4f}")
print(f"F1_score:{f1_score(y_test, y_pred ):.4f}")
```

```
--DETAILS--
Accuracy:0.9561
Precision:0.9459
Recall:0.9859
F1_score:0.9655
```

In [25]:

```
cm = confusion_matrix(y_test, y_pred )
print(f"Confussion_Matrix\n")
print(cm)
```

```
Confussion_Matrix

[[39  4]
 [ 1 70]]
```

In [26]:

```
print("\n--- CLASSIFICATION REPORT---")
print(classification_report(y_test, y_pred, target_names=['tested_positive', 'tested
```

```
--- CLASSIFICATION REPORT---
                precision    recall  f1-score   support

tested_positive       0.97      0.91      0.94        43
tested_negative       0.95      0.99      0.97        71
```

```
           accuracy                           0.96       114
          macro avg       0.96      0.95      0.95       114
       weighted avg       0.96      0.96      0.96       114
```

In [31]:

```
#MLE Estimation

mle_model = LogisticRegression(
    penalty='none',
    solver='lbfgs',
    max_iter=1000
)

mle_model.fit(X_train, y_train)

y_pred_mle = mle_model.predict(X_test)

print(y_pred_mle)
```

```
/usr/local/lib64/python3.6/site-packages/sklearn/utils/validation.py:63: DataConversi
onWarning: A column-vector y was passed when a 1d array was expected. Please change t
he shape of y to (n_samples, ), for example using ravel().
  return f(*args, **kwargs)
[1 0 0 1 1 0 0 0 1 1 1 0 1 0 1 0 1 1 1 0 1 1 0 1 1 1 1 1 1 0 1 1 1 1 1 1 0
 1 0 1 1 0 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 0 1 1 1 0 0 1 1 1 0 0 1 1 0 0 1 0
 1 1 1 1 1 1 0 1 1 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 1 0 0 1 0 0 1 1 1 0 1 1 0
 1 0 0]
/usr/local/lib64/python3.6/site-packages/sklearn/linear_model/_logistic.py:765: Conve
rgenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
```

In [32]:

```
print(f"Accuracy:{accuracy_score(y_test, y_pred_mle ):.4f}")
print(f"Precision:{precision_score(y_test, y_pred_mle ):.4f}")
print(f"Recall:{recall_score(y_test, y_pred_mle ):.4f}")
print(f"F1_score:{f1_score(y_test, y_pred_mle ):.4f}")
```

```
Accuracy:0.9561
Precision:0.9459
Recall:0.9859
F1_score:0.9655
```

In [33]:

```
print("\nIntercept:")
print(mle_model.intercept_)

coef_mle = pd.DataFrame({
    "Feature":X.columns,
    "Coefficient": mle_model.coef_[0]
})

print("\nMLE Coefficients")
print(coef_mle)
```

```
Intercept:
[4.76080087]

MLE Coefficients
                   Feature  Coefficient
0              mean radius     9.902457
```

```
1              mean texture     0.320009
2            mean perimeter    -0.966235
3                 mean area    -0.039173
4           mean smoothness    -2.066118
5          mean compactness    -1.511351
6            mean concavity    -3.405736
7       mean concave points    -4.647494
8             mean symmetry    -2.594562
9    mean fractal dimension     0.197809
10             radius error    -1.009195
11            texture error     3.909032
12          perimeter error    -2.060693
13               area error    -0.008356
14         smoothness error    -0.317433
15        compactness error     2.732336
16          concavity error     4.205747
17     concave points error    -0.216583
18           symmetry error    -0.034718
19  fractal dimension error     0.472280
20             worst radius    -2.914855
21            worst texture    -0.774336
22          worst perimeter     0.434017
23               worst area    -0.010338
24         worst smoothness    -4.022871
25        worst compactness    -2.599392
26          worst concavity    -1.945433
27     worst concave points    -7.013559
28           worst symmetry    -8.195726
29  worst fractal dimension     0.196321
```

In [38]:

```python
map_model1 = LogisticRegression(
    penalty='l1',
    solver='liblinear',
    max_iter=1000,
    random_state=42
)

map_model1.fit(X_train, y_train)

y_pred_map1 = map_model1.predict(X_test)

print(f"Accuracy:{accuracy_score(y_test, y_pred_map1 ):.4f}")

print("\nIntercept:")
print(map_model1.intercept_)

coef_map1 = pd.DataFrame({
    "Feature":X.columns,
    "Coefficient": map_model1.coef_[0]
})

print("\nMLE Coefficients")
print(coef_map1)
```

```
Accuracy:0.9561

Intercept:
[0.]

MLE Coefficients
                   Feature  Coefficient
0              mean radius     4.401313
1             mean texture     0.139012
2           mean perimeter    -0.279094
3                mean area    -0.016807
4          mean smoothness     0.000000
5         mean compactness     0.000000
```

```
6            mean concavity     0.000000
7       mean concave points     0.000000
8             mean symmetry     0.000000
9    mean fractal dimension     0.000000
10             radius error     0.000000
11            texture error     1.708664
12          perimeter error     0.000000
13               area error    -0.099400
14         smoothness error     0.000000
15        compactness error     0.000000
16          concavity error     0.000000
17     concave points error     0.000000
18           symmetry error     0.000000
19  fractal dimension error     0.000000
20             worst radius     0.000000
21            worst texture    -0.426262
22          worst perimeter    -0.026695
23               worst area    -0.014846
24         worst smoothness     0.000000
25        worst compactness     0.000000
26          worst concavity    -3.590885
27     worst concave points     0.000000
28           worst symmetry     0.000000
29  worst fractal dimension     0.000000
```

```
/usr/local/lib64/python3.6/site-packages/sklearn/utils/validation.py:63: DataConversi
onWarning: A column-vector y was passed when a 1d array was expected. Please change t
he shape of y to (n_samples, ), for example using ravel().
  return f(*args, **kwargs)
```

In [40]:

```
#MAP Estimation

map_model2 = LogisticRegression(
    penalty='l2',
    solver='lbfgs',
    max_iter=1000
)

map_model2.fit(X_train, y_train)

y_pred_map2 = map_model2.predict(X_test)

print(f"Accuracy:{accuracy_score(y_test, y_pred_map2 ):.4f}")

print("\nIntercept:")
print(map_model2.intercept_)

coef_map2 = pd.DataFrame({
    "Feature":X.columns,
    "Coefficient": map_model2.coef_[0]
})

print("\nMLE Coefficients")
print(coef_map2)
```

```
/usr/local/lib64/python3.6/site-packages/sklearn/utils/validation.py:63: DataConversi
onWarning: A column-vector y was passed when a 1d array was expected. Please change t
he shape of y to (n_samples, ), for example using ravel().
  return f(*args, **kwargs)
Accuracy:0.9561

Intercept:
[6.53774977]

MLE Coefficients
                  Feature  Coefficient
0             mean radius     2.067478
```

```
1              mean texture     0.098452
2            mean perimeter    -0.144640
3                 mean area     0.001521
4           mean smoothness    -0.231524
5          mean compactness    -0.307308
6            mean concavity    -0.784695
7       mean concave points    -0.551434
8             mean symmetry    -0.291631
9    mean fractal dimension    -0.007795
10             radius error    -0.122845
11            texture error     0.736726
12          perimeter error    -0.073387
13               area error    -0.072385
14         smoothness error    -0.039906
15        compactness error     0.190377
16          concavity error     0.112155
17     concave points error    -0.053847
18           symmetry error    -0.035426
19  fractal dimension error     0.033444
20             worst radius     0.675558
21            worst texture    -0.336908
22          worst perimeter    -0.064251
23               worst area    -0.021222
24         worst smoothness    -0.443176
25        worst compactness    -0.618064
26          worst concavity    -1.545075
27     worst concave points    -0.876362
28           worst symmetry    -1.037857
29  worst fractal dimension    -0.041980
```

```
/usr/local/lib64/python3.6/site-packages/sklearn/linear_model/_logistic.py:765: Conve
rgenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
```

In [41]:

```
data = {'Model' :['MLE','MAP1', 'MAP2'],
        'Accuracy' : [accuracy_score(y_test, y_pred_mle ), accuracy_score(y_test, y_

df = pd.DataFrame(data)
print(df)
```

```
  Model  Accuracy
0   MLE   0.95614
1  MAP1   0.95614
2  MAP2   0.95614
```